

Computer Graphics (CSC214)

► Unit - 02: Scan Conversion Algorithm

Compiled by Shishir Ghimire

Credit Hours: 6 hrs

► TABLE OF CONTENTS

Unit 2: Scan Conversion Algorithm (6 Hrs.)

- 2.1 Scan Converting a Point and a straight Line: DDA Line Algorithm, Bresenham's Line Algorithm
- 2.2 Scan Converting Circle and Ellipse :Mid Point Circle and Ellipse Algorithm
- 2.3 Area Filling: Scan Line Polygon fill Algorithm, Inside-outside Test, Scan line fill of Curved Boundary area, Boundary-fill and Flood-fill algorithm

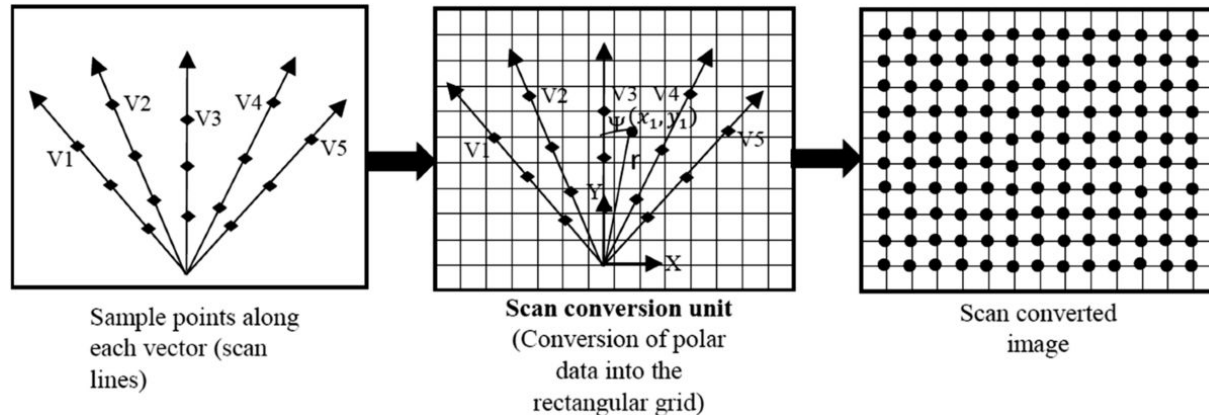
► **Scan Converting a Point and a
Straight Line : DDA ,
Bresenham's Line Algorithm**

2.1

► SCAN CONVERSION:

- Scan Conversion is a procedure used to **digitize or rasterize pixel data** available on frame buffer.
- The process of converting **geometric data** (e.g., lines, circles, ellipses) into **discrete pixels** for display on a raster screen.
- The process of representing continuous **graphics object** as a collection of **discrete pixels** is called scan conversion.

Example: a line is defined by its two end points and the line equation.



► **SCAN CONVERSION of a point:**

Scan conversion of a point requires the two data that are pixel position and color for display.

In C, a point be scan converted using function `putpixel()` defined in library.

Header file is

`<graphics.h>`

`putpixel(x, y, color)`

Here, x & y represent pixel position on 2D display.

► SCAN CONVERSION of a line:

Say $y = mx + b$ be the equation of line with end point (x_1, y_1) and (x_2, y_2) then,

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

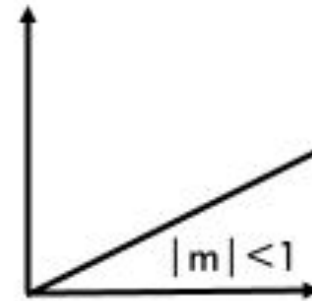
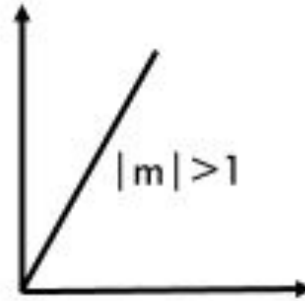
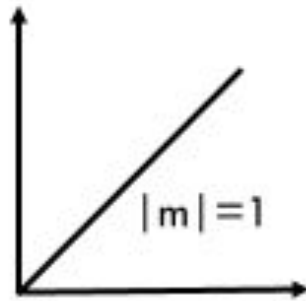
$$\therefore m = \frac{\Delta y}{\Delta x} \dots\dots\dots (i)$$

Here, m represents the slope of line path where by Δx & Δy gives the deflection needed towards horizontal and vertical direction to get new pixel from current pixel position.

- Slope of line also describes the nature and characteristics of line that is going to display.

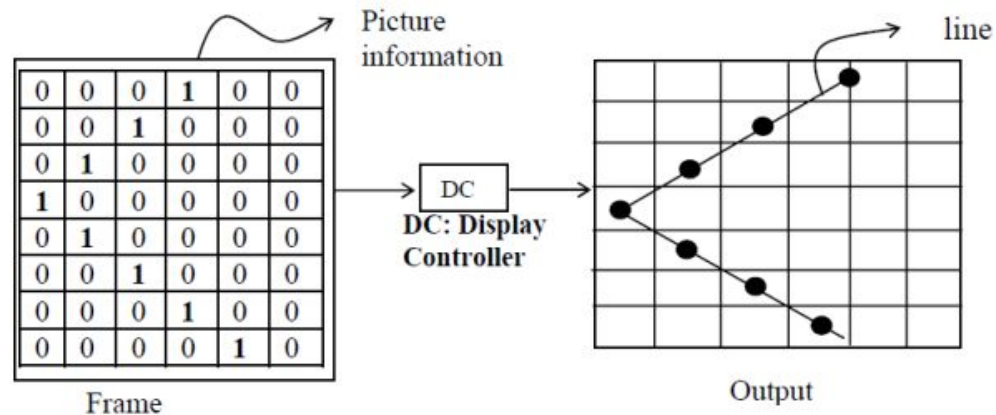
► SCAN CONVERSION of a line:

- A line may have three forms with respect to slope:
 - slope = 1
 - slope > 1
 - slope < 1



► Line Drawing Algorithms:

- ❖ Line drawing algorithms in computer graphics are used to **render lines** on digital displays by determining the appropriate **pixel positions** between **two endpoints**.
- ❖ The primary line drawing **algorithms are**:
 1. Direct Line Drawing i.e $y = mx + c$
 2. Digital Differential Analyzer (DDA) algorithm (Incremental algorithm).
 3. Bresenham's Line Drawing Algorithm.



► DDA Algorithm:

The digital differential analyzer (DDA) is a scan conversion line drawing algorithm based on calculating either x or y from the equation

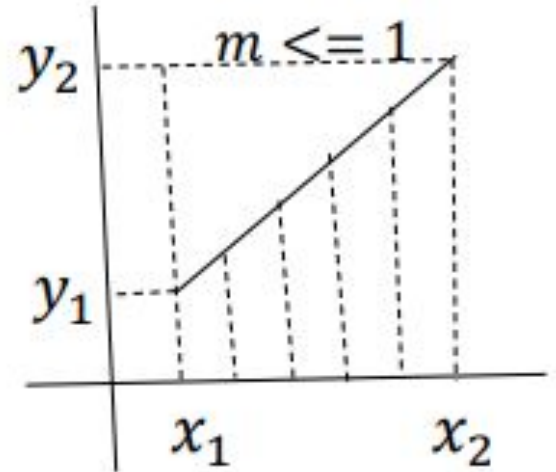
$$\Delta y = m \Delta x$$

The equation of the line is given as:

$$y = mx + b \text{(i)}$$

$$m = (y_2 - y_1)/(x_2 - x_1) \text{ (ii)}$$

We sample the line at unit intervals in one coordinate and determine the corresponding integer values nearest to the line path for the other coordinates.



► DDA Algorithm (Formula Summary):

For any interval Δx , corresponding interval is given by $\Delta y = m\Delta x$.

- **Case 1:** $m < 1$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + m$$

- **Case 2:** $m = 1$,

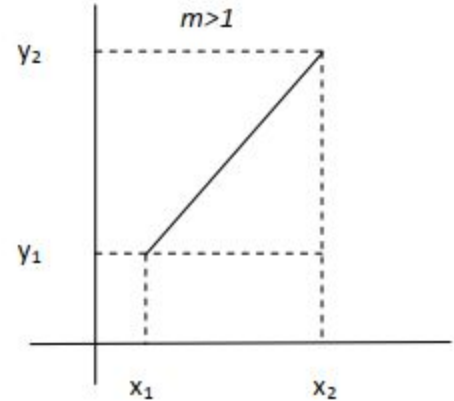
$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + 1$$

- **Case 3:** $m > 1$,

$$x_{k+1} = x_k + 1/m$$

$$y_{k+1} = y_k + 1$$

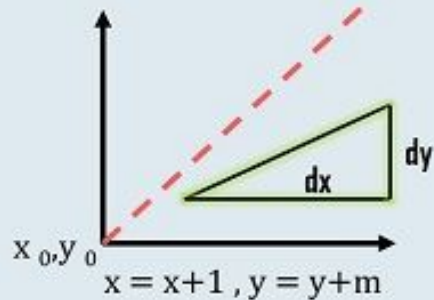


Note: Here k takes value from starting point and increase by 1 until final end point. m can be any real value between 0 and 1.

► DDA Algorithm:

$$m = \frac{y_{k+1} - y_k}{x_{k+1} - x_k}$$

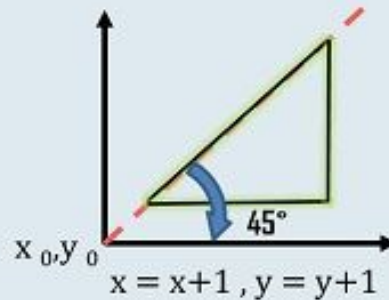
Case 1 – slope $m < 1$



Incrementing x in Unit intervals

$$\begin{aligned}x_{k+1} &= x_k + 1 \\m &= \frac{y_{k+1} - y_k}{1} \\y_{k+1} &= y_k + m\end{aligned}$$

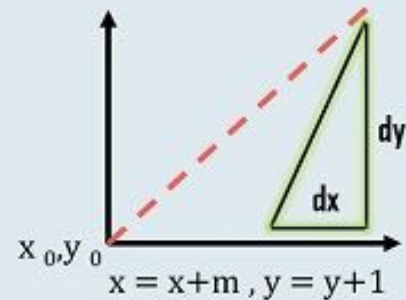
Case 2 – slope $m = 1$



Incrementing x and y in Unit intervals

$$\begin{aligned}x_{k+1} &= x_k + 1 \\y_{k+1} &= y_k + 1\end{aligned}$$

Case 3 – slope $m > 1$



Incrementing y in Unit intervals

$$\begin{aligned}y_{k+1} &= y_k + 1 \\m &= \frac{1}{x_{k+1} - x_k} \\x_{k+1} &= x_k + 1/m\end{aligned}$$

► DDA Algorithm :

- ❖ **Step-1:** Start
- ❖ **Step-2:** Input the line endpoints and store the left endpoint in (x_1, y_1) and right endpoint in (x_2, y_2)
- ❖ **Step-3:** Calculate the value of dx and dy as
 - $dx = x_2 - x_1$
 - $dy = y_2 - y_1$
- ❖ **Step-4:** If $(\text{abs}(dx) > \text{abs}(dy))$ then
 - Steplength = $\text{abs}(dx)$, else
 - Steplength = $\text{abs}(dy)$
- ❖ **Step-5:** Calculate the value of x-increment and y-increment as,
 - $x\text{Increment} = dx / \text{steplength}$
 - $y\text{Increment} = dy / \text{steplength}$
- ❖ **Step-6:** Set $x = x_1$ and $y = y_1$
- ❖ **Step-7:** Draw the pixel ($\text{round}(x)$, $\text{round}(y)$)
- ❖ **Step-8:** k=0 to steplength do
 - $X = x + x\text{Increment}$
 - $Y = y + y\text{Increment}$
 - Draw the pixel ($\text{round}(x)$, $\text{round}(y)$)
- ❖ **Step-9:** End

► DDA Algorithm:

Advantages of DDA:

- It is simple to understand.
- It requires no special skills for implementation.
- It is faster method than direct use of the line equation $y=mx+c$.

Disadvantages of DDA:

- m is stored in floating point number.
- **Accumulation of round-off error** in successive additions can cause calculated pixel positions to drift away from the actual line path for long line segments.
- Rounding operations and floating-point-arithmetic are **time consuming**.

► DDA Algorithm:

C function for DDA algorithm

```
void lineDDA (in x1, int y1, int x2, int y2)
{
    int dx, dy, steps, k;
    float incrx, incry, x, y;
    dx = x2 - x1;
    dy = y2 - y1;
    if (abs(dx) > abs(dy))
        steps = abs(dx);
    else
        steps = abs(dy);
    incrx = dx/steps;
    incry = dy/steps;
    x = x1; /* first point to plot */
    y = y1;
    putpixel(round(x), round(y), 1); //1 is color parameter
    for (k = 1; k <= steps; k++)
    {
        x = x + incrx;
        y = y + incry;
        putpixel(round(x), round(y), 1);
    }
}
```

► DDA Numerical Examples:

Q. Digitized the line with endpoints $(0, 0)$ and $(4, 5)$ using DDA.

► DDA Numerical Examples:

Q. Digitized the line with endpoints (0, 0) and (4, 5) using DDA.

Solution:

Given,

$$(x_1, y_1) = (0, 0)$$

$$(x_2, y_2) = (4, 5)$$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{5 - 0}{4 - 0} = 1.25$$

Since, $m > 1$, from DDA algorithm we have;

$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + \frac{1}{m}$$

x	y	$x\text{-plot}$	$y\text{-plot}$	(x, y)
0	0	0	0	(0, 0)
0.8	1	1	1	(1, 1)
1.6	2	2	2	(2, 2)
2.4	3	2	3	(2, 3)
3.2	4	3	4	(3, 4)
4	5	4	5	(4, 5)

► DDA Numerical Examples:

Q. Digitized the line with endpoints (3, 7) and (8, 3) using DDA..

► DDA Numerical Examples:

Q. Digitized the line with end points (3, 7) and (8, 3) using DDA..

Solution:

Given,

$$(x_1, y_1) = (3, 7)$$

$$(x_2, y_2) = (8, 3)$$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{3 - 7}{8 - 3} = -0.8$$

Since, $m < 1$, from DDA algorithm we have;

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + m$$

x	y	x-plot	y-plot	(x, y)
3	7	3	7	(3, 7)
4	6.2	4	6	(4, 6)
5	5.4	5	5	(5, 5)
6	4.6	6	5	(6, 5)
7	3.8	7	4	(7, 4)
8	3	8	3	(8, 3)

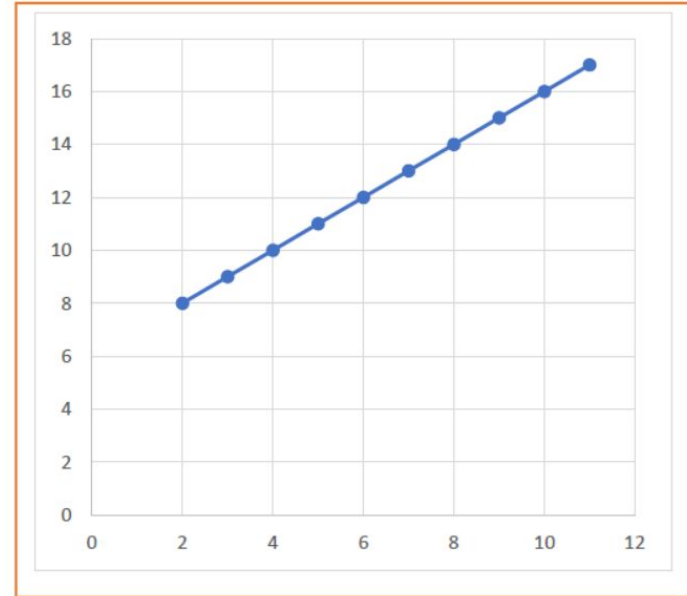
► DDA Numerical Examples:

Calculate the points between the starting point (1, 7) and ending point (11, 17).

As $M = 1$, so case-02 is satisfied.

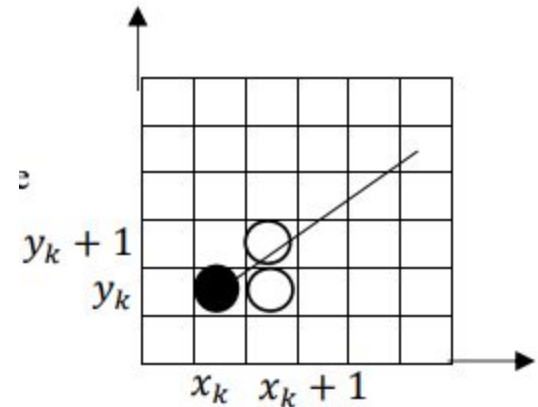
Now, Step-03 is executed until Step-04 is satisfied.

X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
1	7	2	8	(2, 8)
		3	9	(3, 9)
		4	10	(4, 10)
		5	11	(5, 11)
		6	12	(6, 12)
		7	13	(7, 13)
		8	14	(8, 14)
		9	15	(9, 15)
		10	16	(10, 16)
		11	17	(11, 17)



► Bresenham's Line Drawing Algorithm:

- ❖ An accurate and efficient line generating algorithm, developed by Bresenham that scan converts lines **only using integer** calculation to find the next (x, y) position to plot.
- ❖ It **avoids** incremental error accumulation.
- ❖ **Case 1: Line with positive slope less than 1 ($0 < m < 1$)**
- ❖ Pixel position along the line path is determined by sampling at unit x intervals.
- ❖ Starting from left end point, we step to each successive column and plot the pixel **closest to line path**.

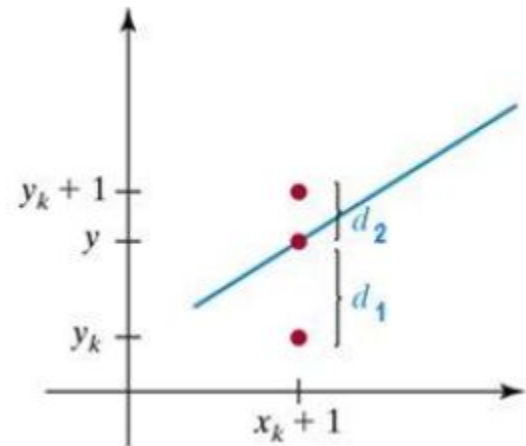


► Bresenham's Line Drawing Algorithm:

Let (x_k, y_k) is pixel at k^{th} step then next point to plot may be either $(x_k + 1, y_k)$ or $(x_k + 1, y_k + 1)$.

Let d_1 & d_2 be the separation of pixel position $(x_k + 1, y_k)$ and $(x_k + 1, y_k + 1)$ from the actual line path.

$$y = mx + b$$



Then at sampling point $(x_k + 1)$:

$$y = m(x_k + 1) + b$$

From figure,

$$d_1 = y - y_k = m(x_k + 1) + b - y_k$$

$$d_2 = (y_k + 1) - y = (y_k + 1) - m(x_k + 1) - b$$

$$\text{Now, } d_1 - d_2 = 2m(x_k + 1) - (y_k + 1) - y_k + 2b = 2m(x_k + 1) - 2y_k + 2b - 1$$

► Bresenham's Line Drawing Algorithm:

A decision parameter p_k for the k^{th} step in the line algorithm can be obtained by rearranging above equation so that it involves only integer calculations. We accomplish this by substituting $m = \frac{\Delta y}{\Delta x}$ in above eqⁿ and defining

$$p_k = \Delta x(d_1 - d_2) = \Delta x[2 \frac{\Delta y}{\Delta x} (x_k + 1) - 2y_k + 2b - 1] = 2\Delta y.x_k - 2\Delta x.y_k + c$$

Where the constant $c = 2\Delta y - \Delta x(2b - 1)$ which is independent of the pixel position. Also, sign of p_k is same as the sign of $d_1 - d_2$.

If decision parameter p_k is negative i.e. $d_1 < d_2$, pixel at y_k is closer to the line path than pixel at $y_k + 1$. In this case we plot lower pixel $(x_k + 1, y_k)$, other wise plot upper pixel $(x_k + 1, y_k + 1)$.

Co-ordinate change along the line occur in unit steps in either x, or y direction. Therefore we can obtain the values of successive decision parameters using incremental integer calculations.

At step $k+1$, decision parameter p_{k+1} is evaluated as.

► Bresenham's Line Drawing Algorithm:

Now, for next step;

$$P_{k+1} = 2\Delta y \cdot x_{k+1} - 2\Delta x \cdot y_{k+1} + c \dots\dots(ii)$$

From (i) & (ii)

$$\therefore P_{k+1} - P_k = 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k)$$

$$\therefore P_{k+1} = P_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k) \quad [\text{Since, } x_{k+1} = x_k + 1]$$

Where, $y_{k+1} - y_k = 0$ or 1

If $P_k < 0$,

$$y_{k+1} = y_k \text{ so } P_{k+1} = P_k + 2\Delta y$$

If $P_k \geq 0$,

$$y_{k+1} = y_k + 1 \text{ so } P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

Therefore, initial decision parameter,

$$P_0 = 2\Delta y \cdot x_0 - 2\Delta x \cdot y_0 + c \quad [\text{from (i)}]$$

$$= 2\Delta y x_0 - 2\Delta x y_0 + 2\Delta y + \Delta x(2b - 1)$$

$$= 2\Delta y x_0 - 2\Delta x y_0 + 2\Delta y + 2b\Delta x - \Delta x$$

$$= 2\Delta y x_0 - 2\Delta x y_0 + 2\Delta y + 2(y_0 - \frac{\Delta y}{\Delta x} x_0)\Delta x - \Delta x$$

$$= 2\Delta y x_0 - 2\Delta x y_0 + 2\Delta y + 2\Delta x y_0 - 2\Delta y x_0 - \Delta x$$

$$= 2\Delta y - \Delta x$$

$$P_0 = 2\Delta y - \Delta x$$

The first decision parameter p_0 is evaluated as.

$$p_0 = 2\Delta y - \Delta x$$

and successively we can calculate decision parameter as

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k)$$

► Bresenham's Algorithm (Formula Summary):

- **Case 1:** $0 < m < 1$

- Initial Decision Parameter $P_0 = 2dy - dx$

- **$P < 0$:**

- $x_{k+1} = x_k + 1$
- $y_{k+1} = y_k$ (No Change)
- $P_{k+1} = P_k + 2dy$

- **$P \geq 0$**

- $x_{k+1} = x_k + 1$
- $y_{k+1} = y_k + 1$
- $P_{k+1} = P_k + 2dy - 2dx$

- **Case 2:** $m \geq 1$

- Initial Decision Parameter $P_0 = 2dx - dy$

- **$P < 0$:**

- $x_{k+1} = x_k$ (No Change)
- $y_{k+1} = y_k + 1$
- $P_{k+1} = P_k + 2dx$

- **$P \geq 0$**

- $x_{k+1} = x_k + 1$
- $y_{k+1} = y_k + 1$
- $P_{k+1} = P_k + 2dx - 2dy$

► Bresenham's Algorithm (Formula Summary):

- **Case 1:** $0 < m < 1$

- Initial Decision Parameter $P_0 = 2dy - dx$

- **$P < 0$:**

- $x_{k+1} = x_k + 1$
- $y_{k+1} = y_k$ (No Change)
- $P_{k+1} = P_k + 2dy$

- **$P \geq 0$**

- $x_{k+1} = x_k + 1$
- $y_{k+1} = y_k + 1$
- $P_{k+1} = P_k + 2dy - 2dx$

- **Case 2:** $m \geq 1$

- Initial Decision Parameter $P_0 = 2dx - dy$

- **$P < 0$:**

- $x_{k+1} = x_k$ (No Change)
- $y_{k+1} = y_k + 1$
- $P_{k+1} = P_k + 2dx$

- **$P \geq 0$**

- $x_{k+1} = x_k + 1$
- $y_{k+1} = y_k + 1$
- $P_{k+1} = P_k + 2dx - 2dy$

Note: For $m = -ve$, replace the sign by $-ve$, $x_{k+1} = x_k - 1$ for $dx = -ve$ and $y_{k+1} = y_k - 1$ for $dy = -ve$

► Bresenham's Line Drawing Algorithm:

Algorithm

1. Input the two line endpoints and store the left endpoint in (x_0, y_0) .
2. Load (x_0, y_0) into the frame buffer i.e. plot the first point.
3. Calculate constants Δx , Δy , $2\Delta y$ and $2\Delta y - 2\Delta x$ and obtain the starting value for the decision parameter as

$$P_0 = 2\Delta y - \Delta x$$

4. At each x_k , along the line, starting at $k=0$, perform the following tests:

If $P_k < 0$, then next point to plot is $(x_k + 1, y_k)$ and

$$P_{k+1} = P_k + 2\Delta y$$

Otherwise, the next point to plot is $(x_k + 1, y_k + 1)$ and

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

5. Repeat step 4 Δx times.

[Note: For $m > 1$, just interchange the role of x & y]

C Implementation

► **BLA:**

```
void lineBresenham (int x1, int y1, int x2, int y2){
    int dx = abs(x2-x1), dy=abs(y2-y1);
    int pk, xEnd;
    pk=2*dy-dx;
    //determine which point to use as start, which as end
    if(x1>x2){
        x = x2;
        y = y2;
        xEnd = x1;
    }
    else {
        x = x1;
        y = y1;
        xEnd = x2;
    }
    putixel (x,y,1);
    while (x < xEnd)
    {
        x++;
        if(pk<0)
            pk=pk+2*dy;
        else
        {
            Y++;
            pk= pk+2*dy-2*dx
        }
        putpixel (x,y,1);
    }
}
```

► Bresenham's Line Drawing Algorithm:

❖ Advantages of Bresenham's line algorithm (BSA) over DDA:

- In DDA algorithm each successive point is computed in floating point, so it required more time and memory space. While in BSA each successive point is calculated in integer value or whole number. So it requires less time and less memory.
- In DDA, since the calculated point value is floating point number, it should be rounded at the end of calculation but in BSA it does not need to round, so there is no accumulation of rounding error.
- Due to rounding error, the line drawn by DDA algorithm is not accurate, while in BSA line is accurate.
- DDA algorithm cannot be used in other application except line drawing, but BSA can be implemented in other application such as circle, ellipse, and other curves.

❖ Disadvantage:

- This algorithm is meant for basic line drawing only Initializing is not a part of Bresenham's line algorithm. So to draw smooth lines, you should want to look into a different algorithm.

► Bresenham's Line Drawing Algorithm Numerical Examples:

Q. Digitize the line with endpoints $(20, 10)$, $(30, 18)$ using Bresenham's line algorithm.

► Bresenham's Line Drawing Algorithm Numerical Examples:

Q. Digitize the line with endpoints (20, 10), (30, 18) using Bresenham's line algorithm.

Solution:

Here,

$$(x_1, y_1) = (20, 10)$$

$$(x_2, y_2) = (30, 18)$$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{18 - 10}{30 - 20} = 0.8 < 1$$

Here, $\Delta x = 10$, $\Delta y = 8$, $2\Delta y = 16$, $2\Delta y - 2\Delta x = -4$

The initial decision parameter (P_0) = $2\Delta y - \Delta x = 16 - 10 = 6 > 0$

Since, for the Bresenham's line drawing algorithm of slope $m < 1$, we have

If $P_k \geq 0$,

$$y_{k+1} = y_k + 1 \text{ \& } x_{k+1} = x_k + 1 \text{ and } P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

If $P_k < 0$,

$$y_{k+1} = y_k \text{ \& } x_{k+1} = x_k + 1 \text{ and } P_{k+1} = P_k + 2\Delta y$$

k	P_k	(x_{k+1}, y_{k+1})
0	6	(21, 11)
1	2	(22, 12)
2	-2	(23, 12)
3	14	(24, 13)
4	10	(25, 14)
5	6	(26, 15)
6	2	(27, 16)
7	-2	(28, 16)
8	14	(29, 17)
9	10	(30, 18)

► Bresenham's Line Drawing Algorithm Numerical Examples:

Q. Digitize the line with endpoints (20, 15) & (30, 30) using Bresenham's line algorithm..

Solution:

Here,

$$(x_1, y_1) = (20, 15)$$

$$(x_2, y_2) = (30, 30)$$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{30 - 15}{30 - 20} = 1.5 > 1$$

Here, $\Delta x = 10$, $\Delta y = 15$, $2\Delta x = 20$, $2\Delta x - 2\Delta y = -10$

The initial decision parameter (P_0) = $2\Delta x - \Delta y = 20 - 15 = 5 > 0$

Since, for the Bresenham's line drawing algorithm of slope $m > 1$, we have

If $P_k \geq 0$,

$$x_{k+1} = x_k + 1 \text{ \& } y_{k+1} = y_k + 1 \text{ and } P_{k+1} = P_k + 2\Delta x - 2\Delta y$$

If $P_k < 0$,

$$x_{k+1} = x_k \text{ \& } y_{k+1} = y_k + 1 \text{ and } P_{k+1} = P_k + 2\Delta x$$

k	P_k	(x_{k+1}, y_{k+1})
0	5	(21, 16)
1	-5	(21, 17)
2	15	(22, 18)
3	5	(23, 19)
4	-5	(23, 20)
5	15	(24, 21)
6	5	(25, 22)
7	-5	(25, 23)
8	15	(26, 24)
9	5	(27, 25)
10	-5	(27, 26)
11	15	(28, 27)
12	5	(29, 28)
13	-5	(29, 29)
14	15	(30, 30)

► Bresenham's Line Drawing Algorithm Numerical Examples:

Q. Digitize a line with endpoints A(6, 12) and B(10, 5) using Bresenham's line drawing.

Solution:

Here,

$$(x_1, y_1) = (6, 12)$$

$$(x_2, y_2) = (10, 5)$$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{5 - 12}{10 - 6} = -1.75$$

$$\therefore |m| = 1.75 > 1$$

Here, $\Delta x = |x_2 - x_1| = 4$, $\Delta y = |y_2 - y_1| = 7$, $2\Delta x = 20$, $2\Delta x - 2\Delta y = -10$

The initial decision parameter (P_0) = $2\Delta x - \Delta y = 8 - 7 = 1 > 0$

Since, for the Bresenham's line drawing algorithm of slope $m > 1$, we have
(Here, $y_1 > y_2$ so y is decremented in successive step)

If $P_k \geq 0$,

$$x_{k+1} = x_k + 1 \text{ \& } y_{k+1} = y_k - 1 \text{ and } P_{k+1} = P_k + 2\Delta x - 2\Delta y$$

If $P_k < 0$,

$$x_{k+1} = x_k \text{ \& } y_{k+1} = y_k - 1 \text{ and } P_{k+1} = P_k + 2\Delta x$$

k	P_k	(x_{k+1}, y_{k+1})
0	1	(7, 11)
1	-5	(7, 10)
2	3	(8, 9)
3	-3	(8, 8)
4	5	(9, 7)
5	-1	(9, 6)
6	7	(10, 5)

► DDA Algorithm vs Bresenham's Line Algorithm:

DDA Algorithm	Bresenham's Line Algorithm
1. DDA Algorithm use floating point, i.e., Real Arithmetic.	1. Bresenham's Line Algorithm use fixed point, i.e., Integer Arithmetic
2. DDA Algorithms uses multiplication & division its operation	2. Bresenham's Line Algorithm uses only subtraction and addition its operation
3. DDA Algorithm is slowly than Bresenham's Line Algorithm in line drawing because it uses real arithmetic (Floating Point operation)	3. Bresenham's Algorithm is faster than DDA Algorithm in line because it involves only addition & subtraction in its calculation and uses only integer arithmetic.
4. DDA Algorithm is not accurate and efficient as Bresenham's Line Algorithm.	4. Bresenham's Line Algorithm is more accurate and efficient at DDA Algorithm.
5. DDA Algorithm can draw circle and curves but are not accurate as Bresenham's Line Algorithm	5. Bresenham's Line Algorithm can draw circle and curves with more accurate than DDA Algorithm.



► Scan Converting Circle & Ellipse: Midpoint Circle and Ellipse Algorithm

2.2

► Circle Drawing Algorithms:

- Circle drawing algorithms in computer graphics are used to render circles on digital displays by determining the appropriate pixel positions that approximate a circular shape.
- The primary **algorithms are**:
 - **Midpoint Circle Algorithm**: Uses symmetry, efficient, and handles integer arithmetic.
 - **Bresenham's Circle Algorithm**: Optimized version of the Midpoint Circle Algorithm

Circle:

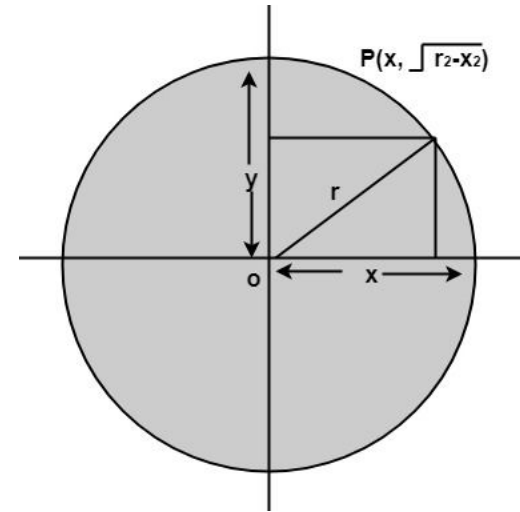
Circle is defined as a set of points that are all at a given distance 'r' from the center position (x_c, y_c) . Equation of circle centered at (x_c, y_c) with radius 'r' is

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

► Circle Drawing Algorithms:

Methods to Draw a Circle:

- Simple Direct Method
- Trigonometric Method (Polar Method)
- Mid-Point Circle Method



Simple Direct Method:

The equation of circle centered at origin and radius r is given by the equation $x^2 + y^2 = r^2$ i.e

• Drawbacks of this method:

- Time consuming – square root and squares computations.
- Non-uniform spacing, due to changing slope of curve.

► Circle Drawing Algorithms:

Polar Method:

If (x,y) be any point on the circle boundary with center at origin and radius r , then

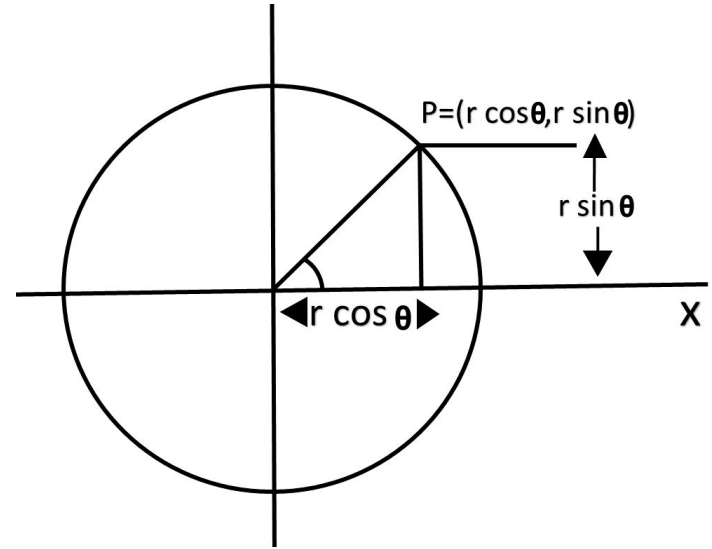
$$x = r \cos \theta$$

$$y = r \sin \theta$$

$$\text{i.e } x,y = (r \cos \theta, r \sin \theta)$$

- Drawbacks of this method:

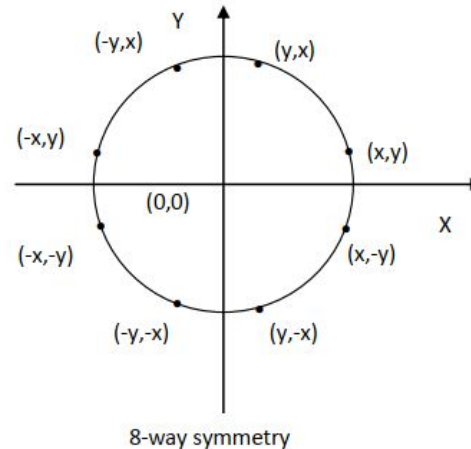
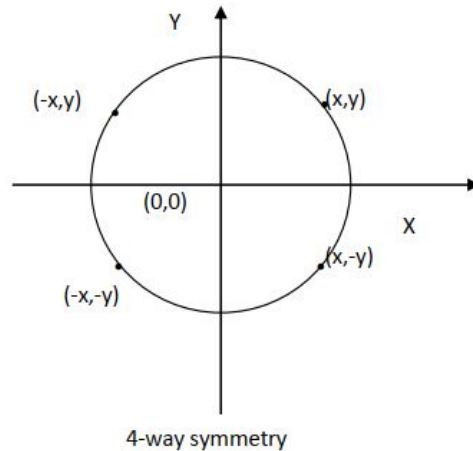
- Floating point calculation.
- Calculation of $\sin \theta$ & $\cos \theta$ is more complex.



► Circle Drawing Algorithms:

Symmetry of a Circle:

- We can **reduce the time** required for **circle generation** by using the symmetries in a circle e.g. 4-way or 8-way symmetry. So we only need to generate the points for one quadrant or octants and then use the symmetry to determine all the other points.
- Problem of computation still persists using symmetry since there are square roots; trigonometric functions are still not eliminated in above algorithms.



► Midpoint Circle Algorithm:

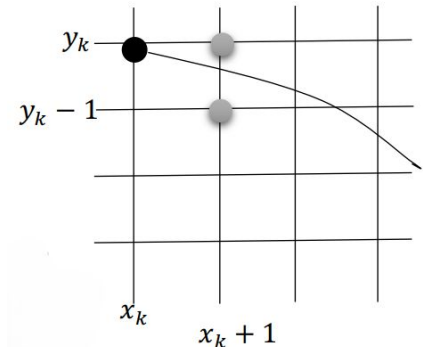
- In mid point circle algorithm, we sample at **unit intervals** and determine the **closest pixel** position to the specified **circle path** at each step.
- For a given radius r , and screen center position (x_c, y_c) , we can first set up our algorithm to calculate pixel positions around a circle path centered at $(0, 0)$ and then each calculated pixel position (x, y) is moved to its proper position by adding x_c to x and y_c to y . (i.e $x=x+x_c, y=y+y_c$)

To apply the mid point method, we define a circle function as:

$$f_{circle} = x^2 + y^2 - r^2$$

To summarize the relative position of point (x, y) by checking sign of f_{circle} function,

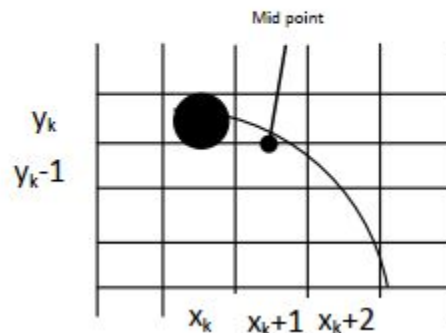
$$f_{circle}(x, y) \begin{cases} < 0, \text{ if } (x, y) \text{ lies inside the circle boundary} \\ = 0, \text{ if } (x, y) \text{ lies on the circle boundary} \\ > 0, \text{ if } (x, y) \text{ lies outside the circle boundary.} \end{cases}$$



► Midpoint Circle Algorithm:

The circle function tests are performed for the mid positions between pixels near the circle path at each sampling step. Thus the circle function is decision parameter in mid point algorithm and we can set up incremental calculations for this functions as we did in the line algorithm.

The figure, shows the mid point between the two candidate pixel at sampling position $x_k + 1$, Assuming we have just plotted the pixel (x_k, y_k) , we next need to determine whether the pixel at position $(x_k + 1, y_k)$ or $(x_k + 1, y_k - 1)$ is closer to the circle.



By evaluating this function at the midpoint between the candidate pixels we can make our decision.

Our decision variable can be defined as:

$$p_k = f\left(x_k + 1, y_k - \frac{1}{2}\right) = (x_k + 1)^2 + \left(y_k - \frac{1}{2}\right)^2 - r^2$$

If $p_k < 0$ the midpoint is inside the circle and the pixel at y_k is closer to the circle
Otherwise, $y_k - 1$ is closer.

► Midpoint Circle Algorithm:

The decision parameter for next position is at $x_{k+1} + 1$ i.e $x_k + 2$.

$$p_{k+1} = f\left(x_{k+1} + 1, y_{k+1} - \frac{1}{2}\right) = (x_k + 2)^2 + \left(y_{k+1} - \frac{1}{2}\right)^2 - r^2$$

Now,

$$p_{k+1} - p_k = (x_k + 2)^2 + \left(y_{k+1} - \frac{1}{2}\right)^2 - r^2 - (x_k + 1)^2 - \left(y_k - \frac{1}{2}\right)^2 + r^2$$

$$p_{k+1} = p_k + x_k^2 + 4x_k + 4 + y_{k+1}^2 - y_{k+1} + \frac{1}{4} - x_k^2 - 2x_k - 1 - y_k^2 + y_k - \frac{1}{4}$$

$$\begin{aligned} p_{k+1} &= p_k + 2x_k + 3 + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) \\ &= p_k + 2x_{k+1} + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1 \end{aligned}$$

Where, y_{k+1} is either y_k or y_{k-1} depending on the sign of p_k .

If $p_k < 0$;

$$y_{k+1} = y_k \Rightarrow p_{k+1} = p_k + 2x_{k+1} + 1$$

If $p_k \geq 0$;

$$y_{k+1} = y_k - 1 \Rightarrow p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

For initial decision parameter;

$$(x_0, y_0) = (0, r)$$

$$p_0 = f\left(1, r - \frac{1}{2}\right) = 1 + \left(r - \frac{1}{2}\right)^2 - r^2 = \frac{5}{4} - r$$

All increments are integer, rounding $\frac{5}{4}$ will give 1 so,

$$p_0 = 1 - r$$

► Midpoint Algorithm (Formula Summary):

Initial Decision Parameter $P_0 = 1 - r$

- **Case 1: $P_k < 0$ (Negative)**

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k \text{ (No Change)}$$

$$P_{k+1} = P_k + 2x_{k+1} + 1$$

- **Case 2: $P_k \geq 0$, (Positive)**

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$P_{k+1} = P_k + 2x_{k+1} - 2y_{k+1} + 1$$

► Midpoint Circle Algorithm:

Algorithm

1. Input radius 'r' and circle center (x_c, y_c) and obtain the first point on the circumference of a circle centered on origin as

$$(x_0, y_0) = (0, r)$$

2. Calculate the initial value of the decision parameter as

$$p_0 = \frac{5}{4} - r$$

3. At each x_k position, starting at $k=0$, perform the following test:

If $p_k < 0$, the next point on circle is $(x_k + 1, y_k)$ and

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Otherwise, the next point on circle is $(x_k + 1, y_k - 1)$ and

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

4. Determine the symmetry point in other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the co-ordinate values,

$$x = x + x_c, \quad y = y + y_c$$

6. Repeat step 3 through 5 until $x \geq y$.

► Midpoint Circle Drawing Algorithm Numerical Examples:

Q. Digitize the circle $x^2 + y^2 = 100$ in first octant.

OR

Calculate the point to draw a circle having radius 10 and center at (0,0).

► Midpoint Circle Drawing Algorithm Numerical Examples:

Q. Digitize the circle $x^2 + y^2 = 100$ in first octant

Solution:

Here,

Center = (0, 0)

Radius (r) = 10

Initial point = (0, r) = (0, 10)

Initial decision parameter $p_0 = 1 - r = 1 - 10 = -9$

From mid-point circle algorithm we have;

If $p_k < 0$;

Plot $(x_k + 1, y_k)$ and $p_{k+1} = p_k + 2x_{k+1} + 1$

$p_k \geq 0$;

Plot $(x_k + 1, y_k - 1)$ and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$

k	p_k	(x_{k+1}, y_{k+1})	$2x_{k+1}$	$2y_{k+1}$
0	-9	(1, 10)	2	20
1	-6	(2, 10)	4	20
2	-1	(3, 10)	6	20
3	6	(4, 9)	8	18
4	-3	(5, 9)	10	18
5	8	(6, 8)	12	16
6	5	(7, 7)	14	14

Now calculate the remaining symmetry points.

► Midpoint Circle Drawing Algorithm Numerical Examples:

Q. Digitize the circle with radius $r = 10$ centered $(3, 4)$ in first octant.

Note: When center is not origin, we first calculate the octants points of the circle in the same way as the center at origin then add the given circle center on each calculated pixel.

Here,

Center = $(3, 4)$

Radius (r) = 10

Initial point = $(0, r) = (0, 10)$

Initial decision parameter $p_0 = 1 - r = 1 - 10 = -9$

From mid-point circle algorithm we have;

If $p_k < 0$;

Plot $(x_k + 1, y_k)$ and $p_{k+1} = p_k + 2x_{k+1} + 1$

$p_k \geq 0$;

Plot $(x_k + 1, y_k - 1)$ and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$

k	p_k	(x_{k+1}, y_{k+1}) at $(0, 0)$	$2x_{k+1}$	$2y_{k+1}$	(x_{k+1}, y_{k+1}) at $(3, 4)$
0	-9	(1, 10)	2	20	(4, 14)
1	-6	(2, 10)	4	20	(5, 14)
2	-1	(3, 10)	6	20	(6, 14)
3	6	(4, 9)	8	18	(7, 13)
4	-3	(5, 9)	10	18	(8, 13)
5	8	(6, 8)	12	16	(9, 12)
6	5	(7, 7)	14	14	(10, 11)

► Midpoint Circle Drawing Algorithm Numerical Examples:

Q. Digitize the circle with radius $r = 5$ centered $(2, 3)$.

Solution:

Here,

Center = $(2, 3)$

Radius (r) = 5

Initial point = $(0, r) = (0, 5)$

Initial decision parameter $p_0 = 1 - r = 1 - 5 = -4$

From mid-point circle algorithm we have;

If $p_k < 0$;

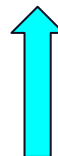
Plot $(x_k + 1, y_k)$ and $p_{k+1} = p_k + 2x_{k+1} + 1$

$p_k \geq 0$;

Plot $(x_k + 1, y_k - 1)$ and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$

Actual pixels

1 st octant	2 nd octant	3 rd octant	4 th octant	5 th octant	6 th octant	7 th octant	8 th octant
(3, 8)	(7, 4)	(-3, 4)	(1, 8)	(1, -2)	(-3, 2)	(7, 1)	(3, -2)
(4, 8)	(7, 5)	(-3, 5)	(0, 8)	(0, -2)	(-3, 1)	(7, 1)	(4, -2)
(5, 7)	(6, 6)	(-2, 6)	(-1, 7)	(-1, -1)	(-2, 0)	(6, 0)	(5, -1)
(6, 6)	(5, 7)	(-1, 7)	(-2, 6)	(-2, 0)	(-1, -1)	(5, -1)	(6, 0)



		1 st			Other pixels (in ...octant)						
k	p_k	(x_{k+1}, y_{k+1})	$2x_{k+1}$	$2y_{k+1}$	2 nd	3 rd	4 th	5 th	6 th	7 th	8 th
0	-4	(1, 5)	2	10	(5, 1)	(-5, 1)	(-1, 5)	(-1, -5)	(-5, -1)	(5, -1)	(1, -5)
1	-1	(2, 5)	4	10	(5, 2)	(-5, 2)	(-2, 5)	(-2, -5)	(-5, -2)	(5, -2)	(2, -5)
2	4	(3, 4)	6	8	(4, 3)	(-4, 3)	(-3, 4)	(-3, -4)	(-4, -3)	(4, -3)	(3, -4)
3	3	(4, 3)	8	6	(3, 4)	(-3, 4)	(-4, 3)	(-4, -3)	(-3, -4)	(3, -4)	(4, -3)

► Ellipse Drawing Algorithms:

- The ellipse is also a symmetric figure like a circle but is **four-way symmetry** rather than eight-way.
- An ellipse is an elongated circle therefore the basic algorithm for drawing ellipse is same as circle computing x and y position at the boundary of the ellipse from the equation of ellipse directly.

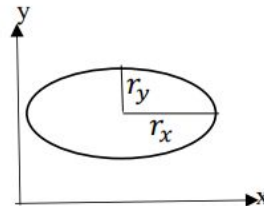
We have equation of ellipse centered at origin (0,0) is

$$\frac{x^2}{r_x^2} + \frac{y^2}{r_y^2} = 1 \text{ which gives}$$

$$y = \pm \frac{r_y}{r_x} \sqrt{(r_x^2 - x^2)} \dots\dots\dots (1)$$

The general equation of ellipse with semi-major axis r_x & semi-minor axis r_y centered at (x_c, y_c) is

$$\left(\frac{x - x_c}{r_x}\right)^2 + \left(\frac{y - y_c}{r_y}\right)^2$$



► Midpoint Ellipse Algorithms:

The mid-point ellipse algorithm decides which point near the boundary (i.e. path of the ellipse) is closer to the actual ellipse path described by the ellipse equation. That point is taken as next point.

- This algorithm is applied to the first quadrant in two parts as in fig Region 1 and Region 2. We process by taking unit steps in x-coordinates direction and finding the closest value for y for each x-step in region 1.
- In first quadrant at region 1, we start at position $(0, r_y)$ and incrementing x and calculating y closer to the path along clockwise direction. When slope becomes -1 then shift unit step in x to y and compute corresponding x closest to ellipse path at Region 2 in same direction.
- Alternatively, we can start at position $(r_x, 0)$ and select point in counterclockwise order shifting unit steps in y to unit step in x when slope becomes greater than -1.

Here, to implement mid-point ellipse algorithm, we take start position at $(0, r_y)$ and step along the ellipse path in clockwise position throughout the first quadrant.

We define ellipse function center at origin i.e. $(x_c, y_c) = (0, 0)$ as

$$f_{\text{ellipse}}(x, y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2$$

$$f_{\text{ellipse}}(x, y) = \begin{cases} < 0, \text{ if } (x, y) \text{ lies inside boundary of ellipse} \\ = 0 \text{ if } (x, y) \text{ lies on the boundary of ellipse} \\ > 0 \text{ if } (x, y) \text{ lies outside the boundary of ellipse} \end{cases}$$

► Midpoint Ellipse Algorithms:

1. Input center (x_c, y_c) and r_x and r_y for the ellipse and obtain the first point as $(x_0, y_0) = (0, r_y)$
2. Calculate initial decision parameter value in Region 1 as

$$P_{10} = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$$

3. At each x_k position, in Region 1, starting at $k = 0$, compute $x_{k+1} = x_k + 1$

If $p_{1k} < 0$, then the next point to plot is

$$p_{1k+1} = p_{1k} + 2r_y^2 x_{k+1} + r_y^2$$

$$y_{k+1} = y_k$$

Otherwise next point to plot is

$$y_{k+1} = y_k - 1$$

$$p_{1k+1} = p_{1k} + 2r_y^2 x_{k+1} + r_y^2 - 2r_x^2 y_{k+1} \quad \text{with } x_{k+1} = x_k + 1 \text{ and } y_{k+1} = y_k - 1$$

4. Calculate the initial value of decision parameter at region 2 using last calculated point say (x_0, y_0) in region 1 as

$$P_{20} = r_y^2 \left(x_0 + \frac{1}{2}\right)^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$$

5. At each y_k position in Region 2 starting at $k = 0$, perform computation $y_{k+1} = y_k - 1$;
if $p_{2k} > 0$, then

$$x_{k+1} = x_k$$

$$p_{2k+1} = p_{2k} - 2r_x^2 (y_k - 1) + r_x^2$$

Otherwise

$$x_{k+1} = x_k + 1$$

$$p_{2k+1} = p_{2k} + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2 \quad \text{where } x_{k+1} = x_k + 1 \text{ and } y_{k+1} = y_k - 1$$

6. Determine the symmetry points in other 3 quadrants.
7. Move each calculated point (x_k, y_k) on to the centered (x_c, y_c) ellipse path as

$$x_k = x_k + x_c;$$

$$y_k = y_k + y_c$$

8. Repeat the process for region 1 until $2r_y^2 x_k \geq 2r_x^2 y_k$ and **Region 2** until $(x_k, y_k) = (x_c, 0)$

► Midpoint Ellipse Algorithm (Formula Summary):

For Region 1:

Initial Decision Parameter $P_{10} = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$

- **Case 1:** $P_{1k} < 0$ (Negative)

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k \text{ (No Change)}$$

$$P_{1k+1} = P_{1k} + 2r_y^2 x_{k+1} + r_y^2$$

- **Case 2:** Otherwise,

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$P_{1k+1} = P_{1k} + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$$

Where $X_{k+1} = X_k + 1$ and $Y_{k+1} = Y_k - 1$

For Region 2:

Initial Decision $P_{20} = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2 r_y^2$

- **Case 1:** $P_{2k} > 0$

$$x_{k+1} = x_k$$

$$y_{k+1} = y_k - 1$$

$$P_{2k+1} = P_{2k} - 2r_x^2 y_{k+1} + r_x^2$$

- **Case 2:** Otherwise

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$P_{2k+1} = P_{2k} + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$$

Where $X_{k+1} = X_k + 1$ and $Y_{k+1} = Y_k - 1$

► Midpoint Ellipse Drawing Algorithm Numerical Examples:

Q. Digitize the ellipse with $r_x = 8$, $r_y = 6$ and center at $(3, 5)$.

Solution:

For region 1

The initial point for the ellipse at origin

$$(x_0, y_0) = (0, r_y) = (0, 6)$$

The initial decision parameter

$$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2 = 6^2 - 8^2 \times 6 + \frac{1}{4} \times 8^2 = -332$$

From midpoint algorithm, for region 1 we know,

If $p1_k < 0$ then

$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k \quad \text{and} \quad p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2$$

If $p1_k \geq 0$ then

$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k - 1 \quad \text{and} \quad p1_{k+1} = p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2.$$

k	$p1_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-332	(1, 6)	72	768
1	-224	(2, 6)	144	768
2	-44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	-108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

Now, we move out of region 1 since $2r_y^2 x_{k+1} > 2r_x^2 y_{k+1}$.

1 st quadrant	2 nd quadrant	3 rd quadrant	4 th quadrant
(4, 11)	(-4, 11)	(-4, -11)	(4, -11)
(5, 11)	(-5, 11)	(-5, -11)	(5, -11)
(6, 11)	(-6, 11)	(-6, -11)	(6, -11)
(7, 10)	(-7, 10)	(-7, -10)	(7, -10)
(8, 10)	(-8, 10)	(-8, -10)	(8, -10)
(9, 9)	(-9, 9)	(-9, -9)	(9, -9)
(10, 8)	(-10, 8)	(-10, -8)	(10, -8)

► Midpoint Ellipse Drawing Algorithm Numerical Examples:

Q. Digitize the ellipse with $r_x = 8$, $r_y = 6$ and center at (3, 5).

For region 2

Initial point for region 2 $(x_0, y_0) = (7, 3)$

The initial decision parameter is

$$\begin{aligned} p2_0 &= r_y^2 \left(x_0 + \frac{1}{2} \right)^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2 \\ &= 36 + (7 + 0.5)^2 + 64 \times 2^2 - 64 \times 36 = -23 \end{aligned}$$

From midpoint algorithm, for region 2 we know,

If $p2_k > 0$ then

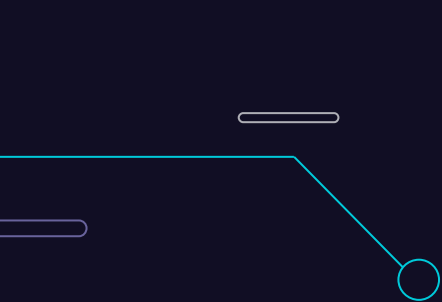
$$x_{k+1} = x_k, \quad y_{k+1} = y_k - 1 \quad \text{and} \quad p2_{k+1} = p2_k + 2r_x^2 y_{k+1} + r_x^2$$

If $p1_k \leq 0$ then

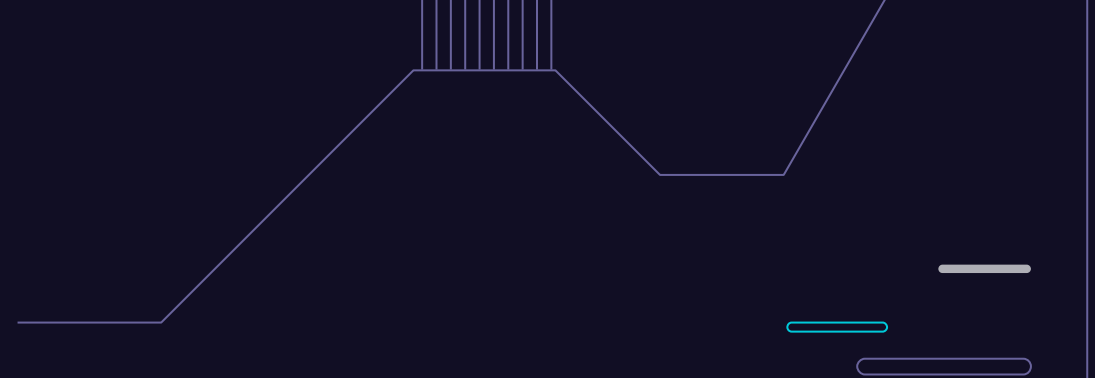
$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k - 1 \quad \text{and} \quad p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2.$$

k	$p2_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-23	(8, 2)	576	256
1	361	(8, 1)	576	72
2	497	(8, 0)	-	-

1 st quadrant	2 nd quadrant	3 rd quadrant	4 th quadrant
(11, 7)	(-11, 7)	(-11, -7)	(11, -7)
(11, 6)	(-11, 6)	(-11, -6)	(11, -6)
(11, 5)	(-11, 5)	(-11, -5)	(11, -5)



► Area Filling



2.3

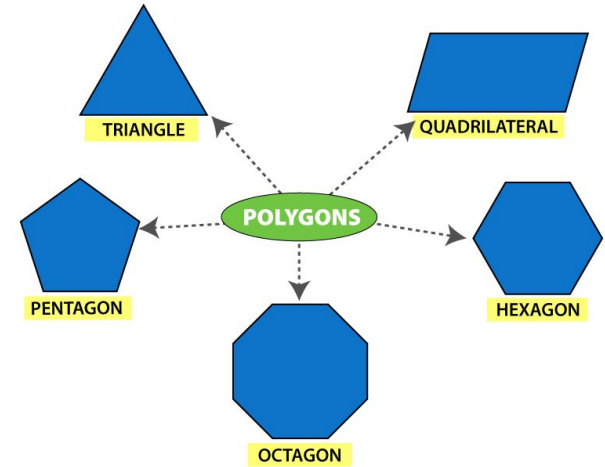
► Area Filling:

A standard output in general graphics is **solid color** or patterned **polygon area**.

Why? Polygons are easier to process since they have **linear boundaries**.

Polygon:

- A Polygon is a closed figure made up of **line segments** (not curves) in a two-dimensional plane. Polygon is the combination of two words, i.e. **poly** (means many) and **gon** (means sides).
- A minimum of three line segments is required to connect end to end, to make a **closed figure**. Thus a polygon with a minimum of three sides is known as Triangle and it is also called 3-gon. An n-sided polygon is called **n-gon**.

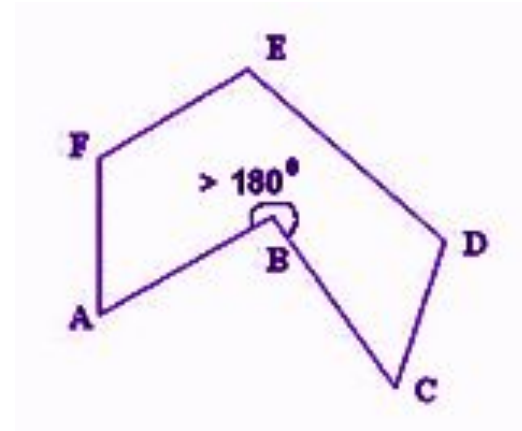
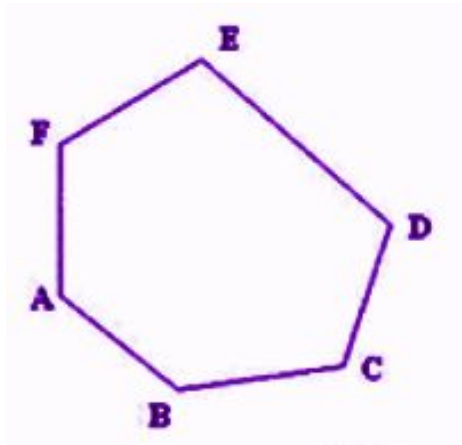


© Byjus.com

► Types of Polygon:

Polygon:

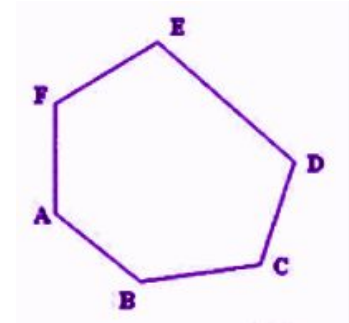
- Based on the the **position** of line segments joining any two points lie, there are two types of Polygon:
 - Convex Polygon
 - Concave Polygon



► Types of Polygon:

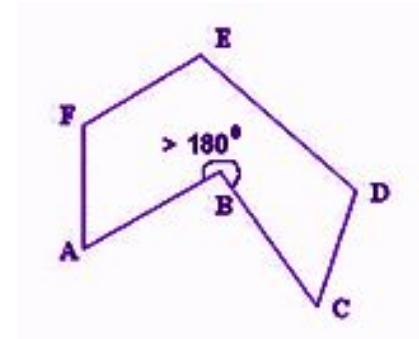
Convex Polygon:

- It is a polygon in which the line segment joining any 2 points within the polygon lies **completely inside** the polygon.
- In a convex polygon, the measure of the **interior angle is less than** 180 degrees.
- The vertices of a convex polygon are always **outwards**.



Concave Polygon:

- It is a polygon in which the line segment joining any 2 points within the polygon **may not lie** completely inside the polygon.
- In a concave polygon, the measure of the interior angle is **more than** 180 degrees.
- The vertices of a concave polygon are **inwards as well as outwards**.



► Filled Area Primitives:

A standard output in general graphics is solid color or patterned polygon area.

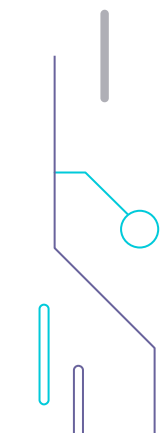
Why? Polygons are easier to process since they have linear boundaries.

- ❖ Filling of polygon with **solid color**.
- ❖ Suppose we want to color a polygon.
 - Coloring must be done **only inside** its boundary.
- ❖ There are **two basic approaches** to area filling in raster systems:
 - One way to fill an area is to **determine the overlap intervals** for scan lines that crosses the area.
 - Another method for area filling is to **start from a given interior position** and point outward from this until a specified boundary is met.

Algorithms for filled area primitives:

1. Scan line polygon fill algorithm
2. Boundary fill algorithm
3. Flood fill algorithm

It works by **intersecting** scan line with polygon edges and **fill** the polygon between pairs of intersection.

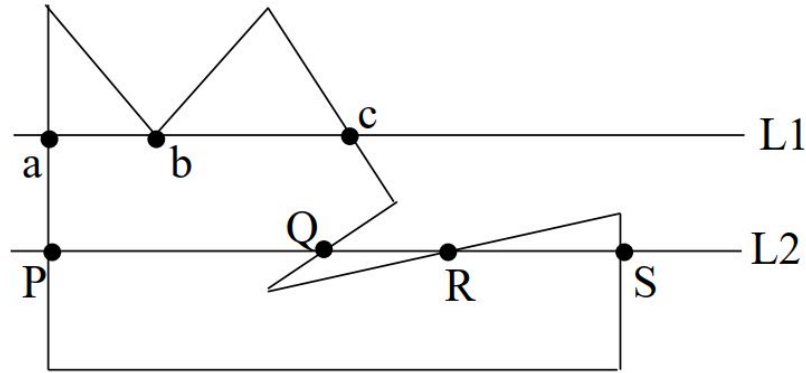


► Scan Line Polygon Fill Algorithm:

The working is depicted by following steps:

- **Step-1:** Find out the $Y_{\min}$ and $Y_{\max}$ from the given polygon.
- **Step-2:** Scan Line intersects with the edge of the polygon from $Y_{\min}$ to $Y_{\max}$. Name each intersection point of the polygon. As per figure they are named as: a, b, c, d.
- **Step-3:** Sort the intersection point from left to right (increasing order of X-Coordinate). i.e (a, b), (b, c), &(c, d).
- **Step-4:** Fill in all pixels between pairs of intersections that lie interior to the polygon. i.e. (a, b) & (c, d).

► Scan Line Polygon Fill Algorithm:



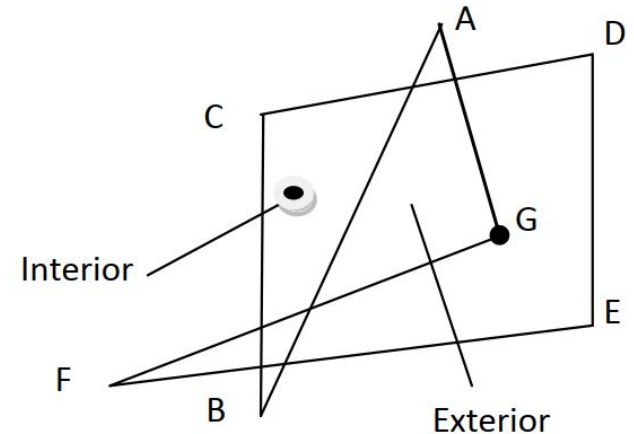
Problem:

- For scan line L1 a, b & c are interior point, therefore we take pairwise points (a, b) & (b,c) and fill all the pixel between these points.
- For scan line L2, we should only take pairwise points (P, Q) & (R, S) because (Q, R) is not part of polygon.
- For scan line L1, we took the vertex 'b' twice i.e.(a, b) & (b, c) but for scan line 'L2' we did not take 'Q' twice.

► Scan Line Polygon Fill Algorithm:

Inside Outside Test:

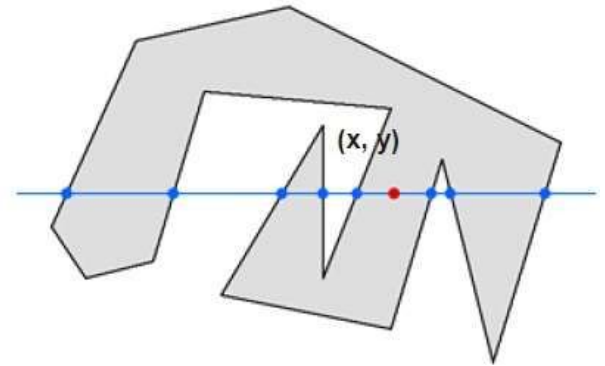
- This test is used to identify whether a **given point** is inside the polygon or outside the polygon.
- It is also called as **Counting Number Method**.
- **Rule:** To identify a point whether it is exterior or interior, draw a line from that point to outside a polygon, if this line **crosses even number of edges** the point is exterior otherwise it is interior.
- But, this method **won't work** if the point is on the edge of the polygon.
- **Methods:**
 - Odd-Even Method
 - Non Zero Winding Number Method



► Scan Line Polygon Fill Algorithm:

Odd-Even Method:

- In this technique, we will count the **edges crossing** along the line of from any point $P(x,y)$ to infinity.
 - If the number of **interactions are odd**, then $P(x,y)$ is an interior point.
 - If the number of **interactions is even**, then point $P(x,y)$ is an exterior point.
- Also called as **odd parity rule** or the **even-odd rule**.
- If Intersection point is a Vertex:
 - Look at other end-points of vertex.
 - Same Side: Even, Different Side: Odd



► Scan Line Polygon Fill Algorithm:

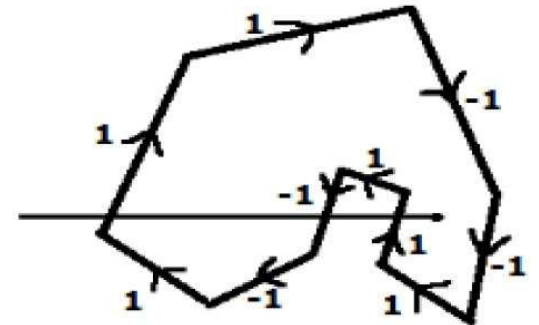
Non Zero Winding Number Method:

- This method is also used with the **simple polygons** to test the given point is interior or not.
- It can be simply understood with the help of a **pin and a rubber band**. Fix up the pin on one of the **edge** of the polygon and **tie-up** the rubber band in it and then **stretch** the rubber band **along the edges** of the polygon.
- When **all the edges** of the polygon are **covered** by the rubber band, check out the pin which has been **fixed up** at the point to be test. If we find **at least one wind** at the point consider it within the polygon, else we can say that the point is **not inside** the polygon.
- **Alternative Method:** Give directions to all the edges of the polygon. Draw a scan line from the point to be test towards the **left most** of X direction.

► Scan Line Polygon Fill Algorithm:

Non Zero Winding Number Method:

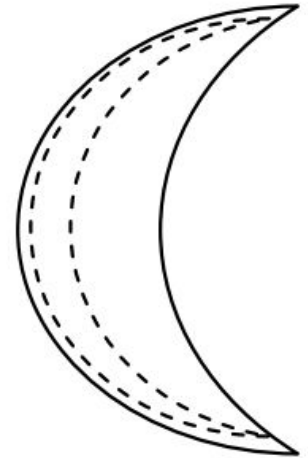
- Give the value 1 to all the edges which are going to **upward direction** and all other -1 as direction values.
- Check the edge direction values from which the scan line is passing and **sum up** them.
- If the **total sum** of this direction value is **non-zero**, then this point to be tested is an interior point, otherwise it is an exterior point.
- In the figure, we sum up the direction values from which the scan line is passing then the **total is $1 - 1 + 1 = 1$** ; which is non-zero. So the point is said to be an interior point.



► Scan Line Polygon Fill Algorithm:

Scan Line Fill of Curved Boundary Area:

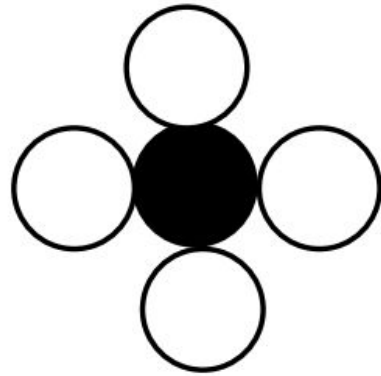
- It requires **more work** than polygon filling, since **intersection calculation** involves nonlinear boundary.
- For simple curves such as **circle or ellipses**, performing a scan line fill is straight forward process. We only need to calculate the two scan-line **intersection on opposite sides** of the curve.
- Then simply fill the **horizontal spans of pixel** between the boundary points on opposite side of curve. Symmetries between quadrants are used to **reduce** the boundary calculation.
- We can fill generating pixel position along curve boundary using **mid-point method**.



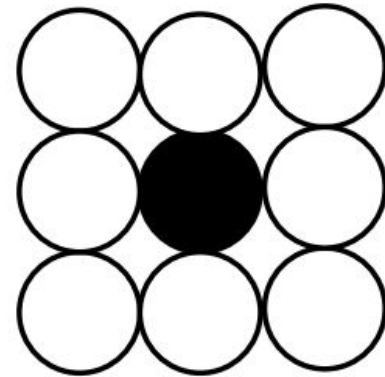
► Boundary Fill Algorithm:

- Boundary Fill Algorithm starts at a **point inside a region** and **paint the interior** towards the **boundary**. If a single color is specified for boundary it proceeds until **boundary color** is reached.
- It accepts **an input**, the coordinate of **interior point $p(x, y)$** , a **fill color** and a **boundary color**.
- Starting from point $p(x, y)$, test is performed to determine whether the **neighboring pixel** is already **filled or boundary** is reached.
- If not, the **neighboring pixels** are filled with **fill color** and their **neighbors are tested**. This process is **repeated** till boundary is reached.
- Two mechanism is used for finding the neighboring pixel:
 - **4-connected** if they are adjacent horizontally and vertically.
 - **8-connected** if they are adjacent horizontally, vertically and diagonally.
- This algorithm is used when boundary is of **single color**. **(For Different Colored boundaries we use Flood Fill Algorithm)**

► Boundary Fill Algorithm:

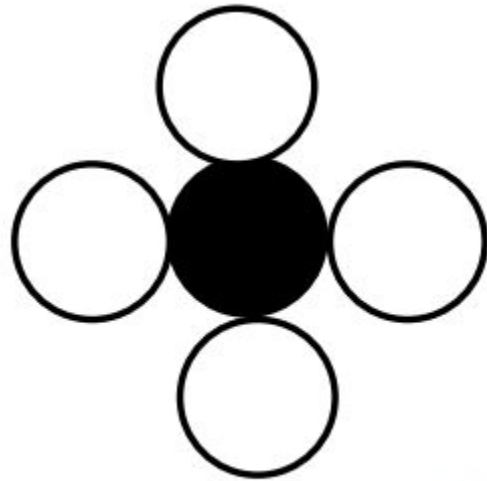


4-connected



8-connected

► Boundary Fill Algorithm:

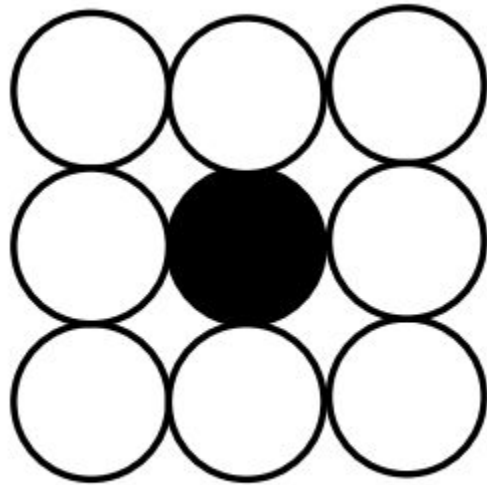


4-connected

Boundary fill 4-connected Algorithm

```
void Boundary_fill4(int x, int y, int b_color, int fill_color)
{
    int value = getpixel(x, y);
    if (value != b_color && value != fill_color)
    {
        putpixel(x, y, fill_color);
        Boundary_fill4(x-1, y, b_color, fill_color);
        Boundary_fill4(x+1, y, b_color, fill_color);
        Boundary_fill4(x, y-1, b_color, fill_color);
        Boundary_fill4(x, y+1, b_color, fill_color);
    }
}
```

► Boundary Fill Algorithm:



8-connected

Boundary fill 8-connected Algorithm

```
void Boundary-fill8(int x,int y,int b_color, int
fill_color)
{
    int current = getpixel (x, y);
    if (current !=b_color && current!=fill_color)
    {
        putpixel (x,y,fill_color);
        Boundary_fill8(x-1, y, b_color,fill_color);
        Boundary_fill8(x+1, y, b_color, fill_color);
        Boundary_fill8(x, y-1, b_color, fill_color);
        Boundary_fill8(x, y+1, b_color, fill_color);
        Boundary_fill8(x-1, y-1, b_color,fill_color);
        Boundary_fill8(x-1, y+1, b_color,fill_color);
        Boundary_fill8(x+1, y-1, b_color,fill_color);
        Boundary_fill8(x+1, y+1, b_color,fill_color);
    }
}
```

► Flood Fill Algorithm:

- This algorithm is used when **boundary is of different color** (fill the area that is not defined within single boundary color).
- If fill area is bounded with **different color**, we can paint that area by replacing a **specified interior color** instead of searching for **boundary color value**.
- We start from a **specified interior pixel (x, y)** and reassign all pixel values that are currently set to a given interior color with **desired fill-color**.
- This algorithm also relies on :
 - **4-connect** method of filling pixels.
 - **8-connect** method of filling pixels.

► Flood Fill Algorithm:

Algorithm (for 4-connected)

```
void flood_fill4(int x, int y, int fill_color, int old_color)
{
    int current = getpixel (x,y);
    if (current==old_color)
    {
        putpixel (x,y,fill_color);
        flood_fill4(x-1, y, fill_color, old_color);
        flood_fill4(x+1, y, fill_color, old_color);
        flood_fill4(x, y-1, fill_color, old_color);
        flood_fill4(x, y+1, fill_color, old_color);
    }
}
```

► Flood Fill Algorithm vs Boundary Fill Algorithm:

Difference between Boundary and Flood fill Algorithm

Boundary fill Algorithm	Flood fill Algorithm
Area filling is started inside a point within a boundary region and fill the region with in the specified color until it reaches the boundary.	Area filling is started from a point and it replaces the old color with the new color.
It is used in interactive packages where we can specify the region boundary.	It is used when we cannot specify the region boundary.
It is less time consuming.	It consumes more time.
It searches for boundary.	It searches for old color.

► PYQs:

- ❖ Generate the points between the endpoints of a line (2,2) and (9,6) by using DDA Line Drawing Algorithm. (5) 2021
- ❖ How can you draw line using Bresenham's Line drawing Algorithm? Trace it to rasterize line with endpoints P(1,4) and Q(7,8).
- ❖ What is a Digital Differential Analyzer (DDA)? How can you draw the line using this algorithm? Trace the points on the line path between A (1,3) and B (6,7) using Bresenham's line drawing algorithm. 2080
- ❖ Explain the importance of filling algorithms in graphics applications. Differentiate between boundary and flood fill algorithm with algorithm. 2080

► PYQs:

- ❖ Write down algorithm steps of mid-point ellipse drawing algorithm. 2080
- ❖ Explain the working details of Midpoint circle algorithm? Trace the points along the line path having two end points (6,9) and (2,3) using Bresenham's line drawing algorithm. 2079
- ❖ Describe the requirement for line clipping. Explain the scan line polygon filling algorithm. 2078
- ❖ Plot the 1st octant of a circle centered at origin, having the radius 10 units. 2078

► Assignment-2 (Page-1):

Due Date: 19th December, 2025

1. Generate the points between the endpoints of a line (2,2) and (9,6) by using DDA Line Drawing Algorithm. (5) 2021
2. How can you draw line using Bresenham's Line drawing Algorithm? Trace it to rasterize line with endpoints P(1,4) and Q(7,8).
3. What is a Digital Differential Analyzer (DDA)? How can you draw the line using this algorithm? Trace the points on the line path between A (1,3) and B (6,7) using Bresenham's line drawing algorithm. 2080
4. Explain the importance of filling algorithms in graphics applications. Differentiate between boundary and flood fill algorithm with algorithm. 2080
5. Write down algorithm steps of mid-point ellipse drawing algorithm. 2080

► Assignment-2 (Page-2):

Due Date: 19th December, 2025

6. Explain the working details of Midpoint circle algorithm? Trace the points along the line path having two end points (6,9) and (2,3) using Bresenham's line drawing algorithm. 2079
7. Describe the requirement for line clipping. Explain the scan line polygon filling algorithm. 2078
8. Plot the 1st octant of a circle centered at origin, having the radius 10 units. 2078
9. Why isn't normal equation of circle used to compute circle?
10. List the major differences between DDA and Bresenham's Line drawing algorithm. Illustrate the DDA algorithm to the line width end points (2, 2) and (9, 6).
11. Plot the ellipse centered at (0, 0) with radius $r_x = 8$ and $r_y = 6$, using mid point ellipse drawing algorithm.



▶ **THANKS!**

Do you have any questions?

theciceerguy@p_m.me

9841893035

ghimires.com.np

